WEEK 2:

1. Aim

To develop a provider-agnostic Large Language Model (LLM) client that provides a common interface to interact with different LLM providers, such as Gemini API and locally hosted Ollama models.

2. Software Requirements

- Python
- Visual Studio Code
- Google Gemini API Key
- Ollama
- Llama 3.2 Model
- Required Python libraries

3. Libraries Used

1. **ABC (Abstract Base Class):** Used to create a common interface for different LLM providers.
2. **Google GenAI:** Used to connect with the Gemini API.
3. **Requests:** Used to send HTTP requests to the locally running Ollama server.
4. **Python-dotenv:** Used to load API keys from the .env file.
5. **Unittest:** Used to perform unit testing of the LLM client.

4. Installation and Setup

Step 1: Install Ollama

Download and install Ollama on the local system.

Step 2: Download the Llama 3.2 Model

Open PowerShell and execute:

```
ollama pull llama3.2
```

Step 3: Install Required Libraries

```
pip install python-dotenv
pip install google-genai
pip install requests
pip install pytest
```

5. Project Structure

Create a folder named `week-2` in Visual Studio Code.

Create the following files:

1. llm_client.py
2. main.py
3. test_client.py
4. requirements.txt
5. .env

6. API Key Configuration

Create a `.env` file and add the Gemini API key:

```
GEMINI_API_KEY=your_actual_api_key
```

7. Python Program

A. llm_client.py

This file defines the common interface and implements Gemini and Ollama providers.

```
from abc import ABC, abstractmethod
from google import genai
import requests

# Common interface
class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass
```

```
# Gemini cloud provider
class GeminiProvider(LLMProvider):

    def __init__(self, api_key):
        self.client = genai.Client(api_key=api_key)

    def generate(self, prompt):
        response = self.client.models.generate_content(
            model="gemini-3.8-flash",
            contents=prompt
        )
        return response.text


# Ollama local provider
class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):
        self.model = model
        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
        response = requests.post(
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=120
        )

        response.raise_for_status()
        return response.json()["response"]


# Common client
class LLMClient:

    def __init__(self, provider):
        self.provider = provider
```

```
    def generate(self, prompt):
        return self.provider.generate(prompt)
```

B. main.py

This file allows the user to select the LLM provider and generate responses.

```
import os
from dotenv import load_dotenv

from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)

load_dotenv(override=True)

provider_name = input(
    "Choose provider (gemini/ollama): "
).strip().lower()

if provider_name == "gemini":

    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        raise ValueError("Gemini API key not found in .env")

    provider = GeminiProvider(api_key)

elif provider_name == "ollama":

    provider = OllamaProvider("llama3.2")

else:
    raise ValueError("Invalid provider selected")

client = LLMClient(provider)

prompt = input("Enter your question: ")
```

```
answer = client.generate(prompt)

print("\nLLM Response:")
print(answer)
```

C. test_client.py

This file performs unit testing using a fake provider.

```
import unittest
from llm_client import LLMClient

# Fake provider for testing
class FakeProvider:

    def generate(self, prompt):
        return "AI stands for Artificial Intelligence."


class TestLLMClient(unittest.TestCase):

    def test_generate(self):

        provider = FakeProvider()
        client = LLMClient(provider)

        result = client.generate("Explain AI")

        self.assertEqual(
            result,
            "AI stands for Artificial Intelligence."
        )


if __name__ == "__main__":
    unittest.main()
```

D. requirements.txt

```
python-dotenv
pytest
```

```
google-genai
requests
```

8. Execution

Run the following commands in the VS Code terminal.

Step 1: Install dependencies

```
pip install -r requirements.txt
```

Step 2: Run the program

```
python main.py
```

Step 3: Select the provider

Enter either:

- gemini
- ollama

Step 4: Enter a prompt

Example:

```
Hi. Help me create a new GPT LLM.
```

The selected provider generates a response and displays it in the terminal.

9. Output

Output 1: Gemini Provider

Select Gemini and enter a prompt.

```
(venv) PS C:\Users\ASUS\OneDrive\Documents\KLH\GENAI\VS Code\Week 2> py main.py
Choose provider (gemini/ollama): gemini
Enter your question: What is Artifical Intelligence. Give me its definition and Examples

LLM Response:
Here is a clear definition of Artificial Intelligence (AI) along with real-world examples.

---

### **Definition of Artificial Intelligence**

**Artificial Intelligence (AI)** is a field of computer science focused on building smart machines and software capable of performing tasks that typically requ
ire **human intelligence**.

These tasks include learning from experience, recognizing patterns, understanding natural language, making decisions, and solving complex problems.

In short: **AI allows computers to think, learn, and act in ways that mimic human cognitive abilities.**

---

### **How AI Works (Briefly)**
AI systems work by combining large amounts of data with fast, iterative processing and intelligent algorithms. This allows the software to learn automatically
from patterns or features in the data (a process known as **Machine Learning**).

---

### **Examples of AI in Daily Life**

AI is already integrated into much of the technology we use every day. Here are key examples categorized by area:

#### **1. Generative AI & Virtual Assistants**
*   **ChatGPT / Claude / Copilot:** AI language models that can write essays, answer questions, draft code, and converse naturally.
*   **Voice Assistants (Siri, Alexa, Google Assistant):** Understand spoken language, answer queries, set reminders, and control smart home devices.
*   **AI Image Generators (Midjourney, DALL-E):** Create realistic images or art from simple text descriptions.
```

Output 2: Ollama Provider

Select Ollama and enter a prompt.

```
(venv) PS C:\Users\ASUS\OneDrive\Documents\KLH\GENAI\VS Code\Week 2> py main.py
Choose provider (gemini/ollama): ollama
Enter your question: What is AI

LLM Response:
AI, or Artificial Intelligence, refers to the development of computer systems that can perform tasks that typically require human intelligence, such as:

1. Learning: AI systems can learn from data and improve their performance over time.
2. Problem-solving: AI systems can analyze data, identify patterns, and make decisions.
3. Reasoning: AI systems can draw conclusions and make logical deductions.
4. Perception: AI systems can interpret and understand data from sensors, such as images and speech.

There are several types of AI, including:

1. **Narrow or Weak AI**: This type of AI is designed to perform a specific task, such as facial recognition or language translation.
2. **General or Strong AI**: This type of AI is designed to perform any intellectual task that a human can, such as reasoning, problem-solving, and learn
ing.
3. **Superintelligence**: This type of AI is significantly more intelligent than the best human minds, and is capable of solving complex problems that ar
e beyond human capabilities.

AI can be applied in various fields, including:

1. **Virtual assistants**: AI-powered assistants, such as Siri, Alexa, and Google Assistant, can perform tasks such as answering questions, setting remin
ders, and controlling smart home devices.
2. **Image and speech recognition**: AI-powered systems can recognize and analyze images and speech, and can be used in applications such as self-driving
 cars, surveillance systems, and voice assistants.
3. **Predictive analytics**: AI-powered systems can analyze data and make predictions about future events, such as stock prices, weather patterns, and cu
stomer behavior.
4. **Robotics**: AI-powered robots can perform tasks such as assembly, inspection, and maintenance, and can be used in industries such as manufacturing,
logistics, and healthcare.

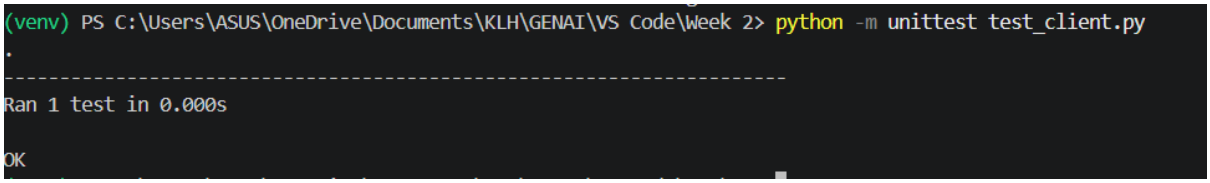
The benefits of AI include:

1. **Increased efficiency**: AI can automate repetitive and mundane tasks, freeing up human time and resources.
2. **Improved accuracy**: AI can analyze large amounts of data and make predictions or decisions with a high degree of accuracy.
3. **Enhanced customer experience**: AI-powered systems can provide personalized recommendations and customer service.
4. **Increased productivity**: AI can help humans to work more efficiently and effectively, leading to increased productivity and competitiveness.
```

Output 3: Unit Testing

Execute the test program:

```
python -m unittest test_client.py
```



4

10. Observations

1. Successfully installed and configured Ollama in the local system.
2. Downloaded the Llama 3.2 model using Ollama.
3. Created a common interface using an abstract base class.
4. Implemented Gemini and Ollama as separate LLM providers.
5. Observed that the same client interface can communicate with different LLM providers.
6. Successfully generated responses using both cloud-based and locally hosted models.
7. Performed unit testing using a fake provider and verified the expected output.

11. Result

Successfully developed a provider-agnostic LLM client that supports both Gemini API and locally hosted Ollama models through a common interface. The program generated responses successfully and passed the unit test.